

1. Anisotropic Oriented-Filter Edge Detection

1.1. Motivation

Classical edge detectors such as Sobel or Canny apply a fixed, isotropic gradient operator to every pixel. While effective for many tasks, they treat all orientations identically at the convolution stage, relying on post-hoc arctangent computation to recover edge direction. An anisotropic oriented-filter approach reverses this order. It evaluates a bank of pre-rotated stencils, each tuned to a specific edge orientation, and selects the one that responds most strongly. The result is a per-pixel gradient magnitude paired with a discrete edge angle, obtained directly from the filter that best matches the local image structure.

The principal advantage is that each stencil can encode a richer model of what an edge looks like at its particular orientation. Polynomial fitting, Gaussian line weighting, and pseudoinverse extraction are all folded into a single set of precomputed weights, so the per-pixel cost at runtime reduces to a weighted sum over a modest number of support pixels.

1.2. The Fused Stencil Filter

The filter operates at a target pixel (X_0, Y_0) by sweeping through N_s candidate orientations indexed by $k = 0, 1, \dots, N_s - 1$. For each orientation θ_k , a stencil of N'_k unique pixel positions surrounds the target. The response for orientation k is the inner product of precomputed weights with image intensities at those positions.

$$R_k = \sum_{\ell=1}^{N'_k} \alpha_{k,\ell} \cdot f(X_0 + \tilde{\delta}_\ell^x, Y_0 + \tilde{\delta}_\ell^y)$$

The winning orientation is then selected by taking the largest absolute response across all candidate angles.

$$k^* = \arg \max_k |R_k|$$

The output at each pixel is the gradient magnitude $|R_{k^*}|$ and the corresponding edge angle θ_{k^*} .

1.3. Variable Definitions

The symbols appearing in the equations above are defined as follows.

R_k is the filter response for orientation k , a single scalar measuring edge strength in the direction θ_k . The index k runs over all candidate orientations, so $k = 0, 1, \dots, N_s - 1$, where N_s is the total number of orientations sampled (commonly 36 for five-degree spacing). N'_k is the number of unique pixel positions in the fused stencil for orientation k after deduplication. For a typical configuration this is on the order of 264 positions. The index ℓ labels each unique pixel position within the stencil, running from 1 to N'_k .

$\alpha_{k,\ell}$ is the precomputed fused weight for stencil position ℓ at orientation k . This single number encodes all the polynomial fitting, pseudoinverse extraction, and Gaussian line weighting that would otherwise be computed at runtime. $f(\cdot)$ denotes the image brightness function, where $f(x, y)$ returns the grayscale intensity at pixel (x, y) . (X_0, Y_0) are the coordinates of the target pixel, the pixel for which the edge strength is being computed. $(\tilde{\delta}_\ell^x, \tilde{\delta}_\ell^y)$ are the horizontal and vertical offsets of stencil position ℓ relative to the target pixel.

k^* is the winning orientation index, defined as $\arg \max_k |R_k|$, meaning the value of k that makes $|R_k|$ largest. The absolute value $|R_k|$ is used because edges can be bright-to-dark or dark-to-bright, and only the magnitude matters for detecting the presence of an edge.

1.4. Demonstration

The filter was applied to a grayscale photograph using 36 orientations ($N_s = 36$), a stencil half-width of 15 pixels, and a Gaussian scale parameter $\sigma = 2.0$. The figure below shows the original image, the recovered gradient magnitude $|R_{k^*}|$, and a hue-coded orientation map where colour encodes θ_{k^*} and brightness encodes edge strength.

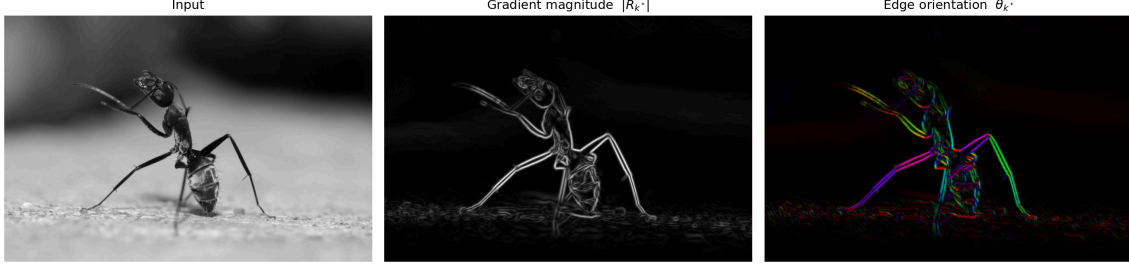


Figure 1: Anisotropic edge detection applied to a test image. Left: grayscale input. Centre: gradient magnitude $|R_{k^*}|$. Right: orientation map (θ_{k^*} encoded as hue, $|R_{k^*}|$ as brightness).

The oriented filter cleanly resolves edges at all angles, including fine structures such as legs and antennae, where isotropic operators tend to blur responses across adjacent orientations.

1.5. Implementation Notes

The demonstration code constructs each oriented kernel as the product of a Gaussian envelope along the line direction and a first-derivative-of-Gaussian profile across it. The per-orientation response is computed via `scipy.ndimage.convolve`, and the winning orientation is selected with a pointwise `argmax` across the response stack. A full production implementation would replace the explicit convolution loop with the precomputed fused stencil weights $\alpha_{k,\ell}$ described above, reducing memory traffic and eliminating redundant floating-point operations.

The Python script `aniso_edge_demo.py` in this directory reproduces all figures shown here.